



Dayananda Sagar University

School of Engineering

Department of Computer Science and Technology



IntentCloud: Intent-Aware Cognitive Cloud Memory System

Under the Guidance of

Dr.Ramandeep Kaur
Dept of CST

Presented By

Putta Sujith	(ENG23CT0058)
K Vikas Aneesh Reddy	(ENG23CT0052)
Karnati Mokshith	(ENG23CT0053)



Dayananda Sagar University

School of Engineering



Table of Contents

- Abstract
- Introduction
- Literature Survey (Detailed Existing System Analysis)
- Problem Statement
- Objectives
- Proposed Methodology



Dayananda Sagar University

School of Engineering



Abstract

- Traditional file storage systems rely on filenames, folders, and keyword-based search, making document retrieval difficult when users cannot remember exact file details.
- Existing AI-powered retrieval solutions often depend on cloud services, leading to privacy concerns, recurring costs, and limited offline accessibility.
- IntentCloud is a self-hosted, intent-aware cognitive cloud memory system that enables users to retrieve documents using natural language queries instead of exact keywords.
- The proposed system combines local AI-based intent understanding, semantic embeddings, and vector similarity search to improve the relevance and accuracy of file retrieval.
- A Raspberry Pi provides secure edge storage, while Cloudflare Tunnel enables protected remote access without exposing the local network.
- The proposed architecture aims to deliver privacy-preserving, cost-effective, and efficient semantic file retrieval on consumer-grade hardware.



Introduction

- Existing storage systems organize files using filenames and folders.
- Users often forget filenames after several months.
- Searching by keyword may return many unrelated files.
- Semantic search understands the context of a query.
- IntentCloud works like a digital memory assistant.
- It retrieves documents based on user intent instead of exact words.
- The system is designed to be private, secure, and self-hosted.
- Existing semantic retrieval solutions do not simultaneously provide intent-aware retrieval, local AI processing, secure storage, and remote accessibility.



Dayananda Sagar University

School of Engineering

Literature Survey



Sl. NO.	Paper Title	Methodology & Accuracy	Pro's	Con's
1	Semantic Search in a Document Management System Using Retrieval-Augmented Generation (Krishnan & Subramanian, 2025)[1]	Uses RAG to understand the user's question and search documents by meaning instead of keywords. Gives accurate results but depends on cloud AI.	Finds relevant documents faster than normal keyword search.	Needs paid cloud APIs, so user data is not completely private.
2	Constructing ChatGPT for PDF Files with LangChain (Jacob et al., 2024)[2]	Splits PDF into small parts, converts them into embeddings, and uses ChatGPT to answer user questions. Works well for document search.	Easy to search and understand PDF content using natural language.	Breaking documents into small chunks may miss the complete context of the document.
3	Blended RAG: Improving RAG Accuracy with Semantic Search and Hybrid Retrieval (Sawarkar et al., 2024)[3]	Combines keyword search and semantic search together to improve retrieval accuracy.	Better search accuracy because it uses two search methods together.	Needs a large amount of memory and powerful hardware for big datasets.
4	Efficient RAG with Intent-Aware Retrieval and Semantic-Preserving Chunking (Puspitasari et al., 2026)[4]	Understands the user's intent before searching and stores document chunks carefully to improve retrieval quality.	Gives more relevant search results by understanding user intention.	Works well only when information is inside nearby chunks; struggles with information spread across a document.
5	Research on an Intelligent File Query System Integrating Natural Language Understanding and Information Retrieval (Nan et al., 2026)[5]	Uses Natural Language Understanding (NLU) and semantic retrieval to find files from user queries. Provides very high search accuracy.	Very accurate file retrieval using AI and semantic understanding.	Requires expensive GPUs and high-end hardware, making it difficult for normal users.



Problem Statement & Objectives

Problem Statement

- Students, researchers, and professionals often struggle to locate documents because they cannot remember exact filenames, folder locations, or keywords.
- As the number of stored files increases, traditional keyword-based search becomes inefficient, leading to longer search times and reduced productivity.
- Existing semantic retrieval systems often depend on cloud services, resulting in privacy concerns, recurring subscription costs, and limited offline accessibility.
- Current solutions do not provide a complete self-hosted, intent-aware document retrieval system that combines privacy, secure storage, and remote accessibility on consumer-grade hardware.
- Therefore, there is a need for a privacy-preserving, self-hosted, intent-aware semantic file retrieval system that enables users to search documents using natural language while ensuring secure and efficient access.



Dayananda Sagar University

School of Engineering



Problem Statement & Objectives

Objectives

- To design and implement a hybrid retrieval pipeline combining local intent parsing (Phi-3 Mini), semantic embeddings, and keyword-based matching capable of surfacing the correct file within the top-3 results for at least 85% of natural-language queries, tested on a corpus of 150+ mixed PDF/DOCX/TXT files. This directly targets the narrow, closed-set intent-classification limitation.
- To ensure the entire pipeline text extraction, embedding generation, intent parsing, and reranking runs fully on local hardware with zero calls to third-party LLM APIs, addressing the privacy and recurring-cost concerns raised by cloud-dependent RAG systems.
- To deliver the original requested file, rather than a generated summary, within roughly 15 seconds end-to-end on CPU-only consumer hardware. This overcomes both the "answer, not file" limitation seen in and the GPU-cluster dependency reported for high-accuracy systems.
- To deploy the complete system upload, indexing, hybrid retrieval, and serving as a single self-hosted service reachable through one public HTTPS URL via Cloudflare Tunnel, since foundational work provides only algorithms, not a deployable system, and locally-hosted systems such as remain confined to a single machine with no remote access.
- To empirically benchmark the hybrid, reranked pipeline against a plain keyword search and a dense-only semantic search on the same test set, so that the improvement from each added component (hybrid retrieval, reranking) is quantified rather than assumed.



Proposed Methodology

Major Features of the Proposed System

- Natural Language Search – Retrieve files using conversational queries.
- Intent-Aware Retrieval – Understands user intent beyond keywords.
- Semantic Search – Finds documents based on meaning using AI embeddings.
- Local AI Processing – Performs inference offline using Phi-3 Mini.
- Secure Edge Storage – Stores files securely on Raspberry Pi.
- Remote Access – Provides secure access through Cloudflare Tunnel.
- Privacy-Preserving – Eliminates dependence on third-party cloud APIs.



Dayananda Sagar University

School of Engineering



Proposed Methodology

Phase 1 - Data Ingestion & Preprocessing

- User uploads PDF / DOCX / TXT through the browser UI.
- Text extraction via PyMuPDF (PDF) and python-docx (DOCX) — fully offline, no per-document cost, avoiding the API dependency .

Phase 2 - Semantic Representation & Indexing

- Extracted text is converted to embeddings using sentence-transformers (all-MiniLM-L6), built on the attention mechanism established.
- Index: Qdrant (embedded/local mode), replacing the originally proposed FAISS index. Justification:
- Qdrant persists vectors and metadata (filename, topic tags, upload time) together, removing the need for a separate metadata store that a bare FAISS index would require.
- It has native hybrid search (dense vector + sparse/keyword scoring) and payload filtering built in, which Phase 4 depends on — FAISS alone only does dense similarity search and has no keyword-matching layer.
- It runs embedded on the Raspberry Pi without a separate server process, keeping the "consumer-grade hardware" constraint intact, and avoids the ~50GB-scale memory footprint reported for hybrid indexes at large corpus sizes in acceptable at our target scale of 150–200 files.

Phase 3 — Intent-Aware Query Understanding

- The natural-language query is parsed locally by Phi-3 Mini (via Ollama) into structured intent (topic/keyword JSON).
- Parsing happens on-device, avoiding the cloud dependency and generalizing beyond the closed-set, four-category classification.

Phase 4 — Hybrid Retrieval & Reranking (replaces plain dense-only FAISS search)

- Dense retrieval: semantic similarity search over embeddings returns the top-N semantically closest candidates.
- Sparse/keyword retrieval: a BM25-style match runs in parallel, so an exact filename or keyword hit is never outranked by a merely topical one — directly targeting the failure mode observed in the earlier dense-only RAG attempt, and consistent with the accuracy gains from combining retrieval strategies reported in [8]. Both retrieval modes are run through Qdrant's native hybrid-query support, which fuses dense and sparse scores in a single query rather than requiring two separate systems to be merged manually — this is the specific reason Qdrant was chosen over FAISS in Phase 2.
- Reranking: the merged candidate set is reordered by a lightweight local cross-encoder before the top-3 are shown, improving relevance beyond raw vector distance and providing the human-interpretable "why this matched" explanation missing.



Dayananda Sagar University

School of Engineering



Proposed Methodology

Phase 5 — Secure Storage & Remote Delivery

- Files, metadata, and the vector index persist on the Raspberry Pi 4B + USB storage (the edge node).
- Cloudflare Tunnel exposes one HTTPS endpoint — no port forwarding, no static IP — closing the "confined to a single machine, no remote access" gap in .

Phase 6 — Evaluation Plan

- Test corpus: 150–200 files spanning 5–8 topic categories.
- Query set: 30–50 natural-language queries, each with a known ground-truth target file, authored independently of the upload process to avoid bias.
- Metrics:
 - Top-3 retrieval accuracy (%) — supports O1
 - Average end-to-end query latency (s), including reranking — supports O3
 - Accuracy delta vs. keyword-only baseline and vs. dense-only baseline — supports O5
- Reporting: results presented as a comparison table/chart in the final review, directly answering "how do you know hybrid retrieval actually improved things?"

Proposed Methodology

Proposed IntentCloud System Architecture

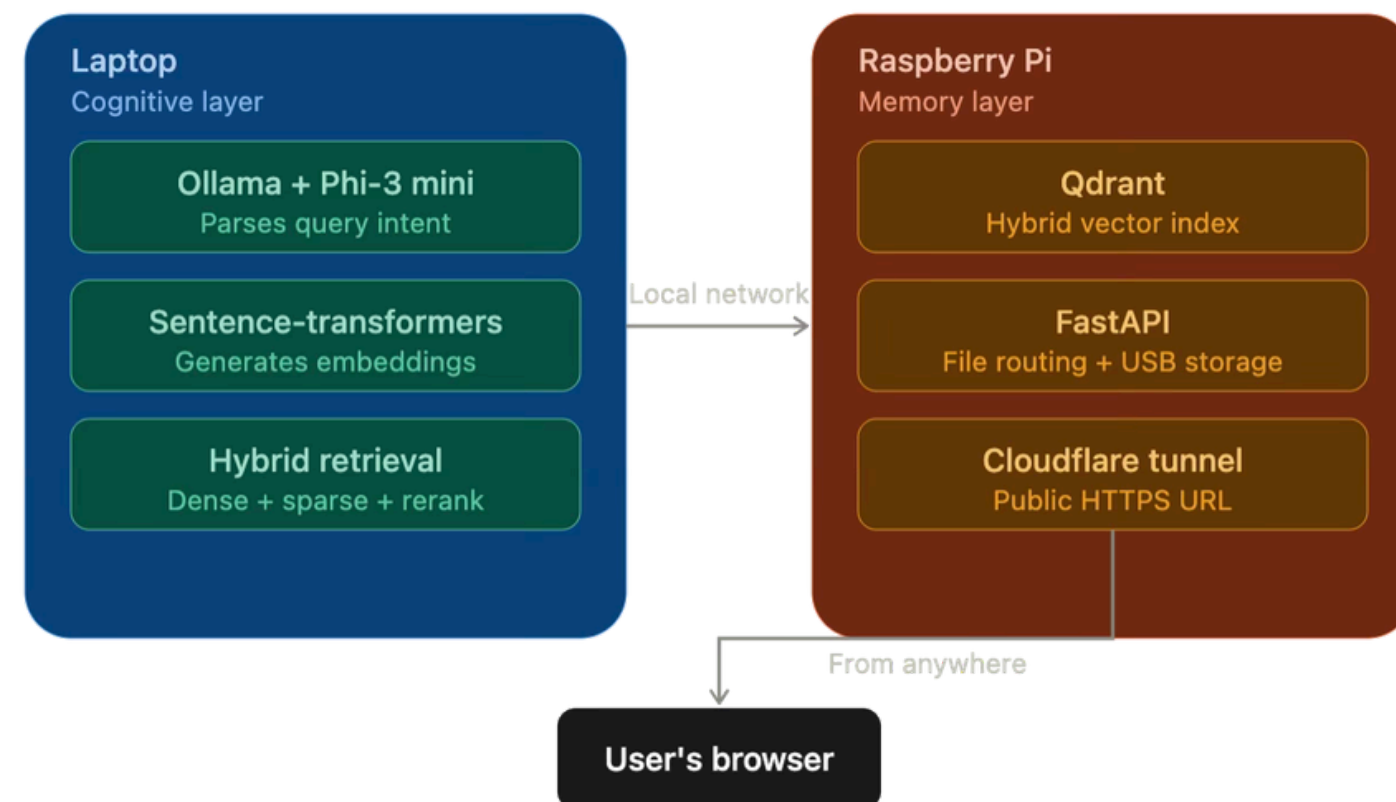


Figure 1. Proposed IntentCloud architecture showing the interaction between the User Interface, Cognitive Layer, Memory Layer, and Cloud Access

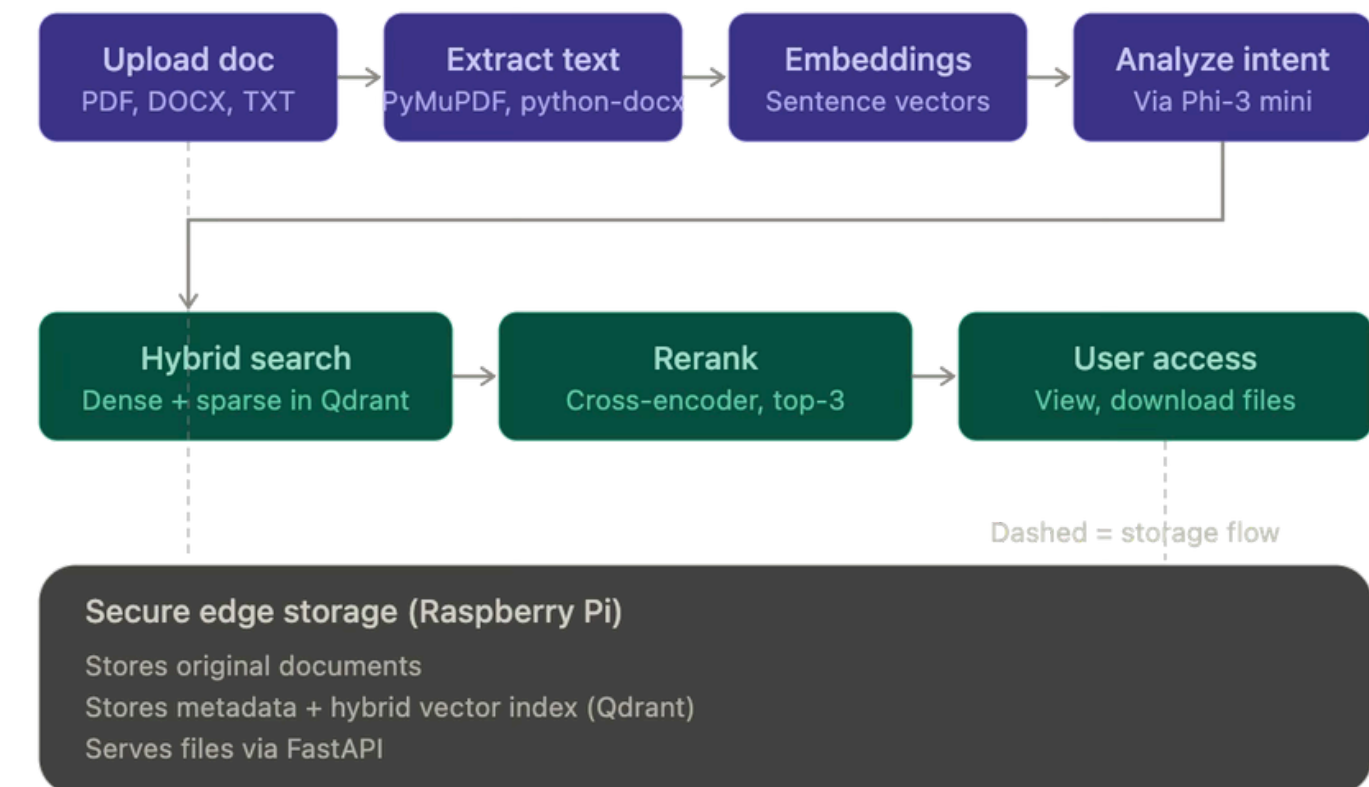


Figure 2 .Workflow of IntentCloud for intent-aware semantic file retrieval.



Dayananda Sagar University

School of Engineering

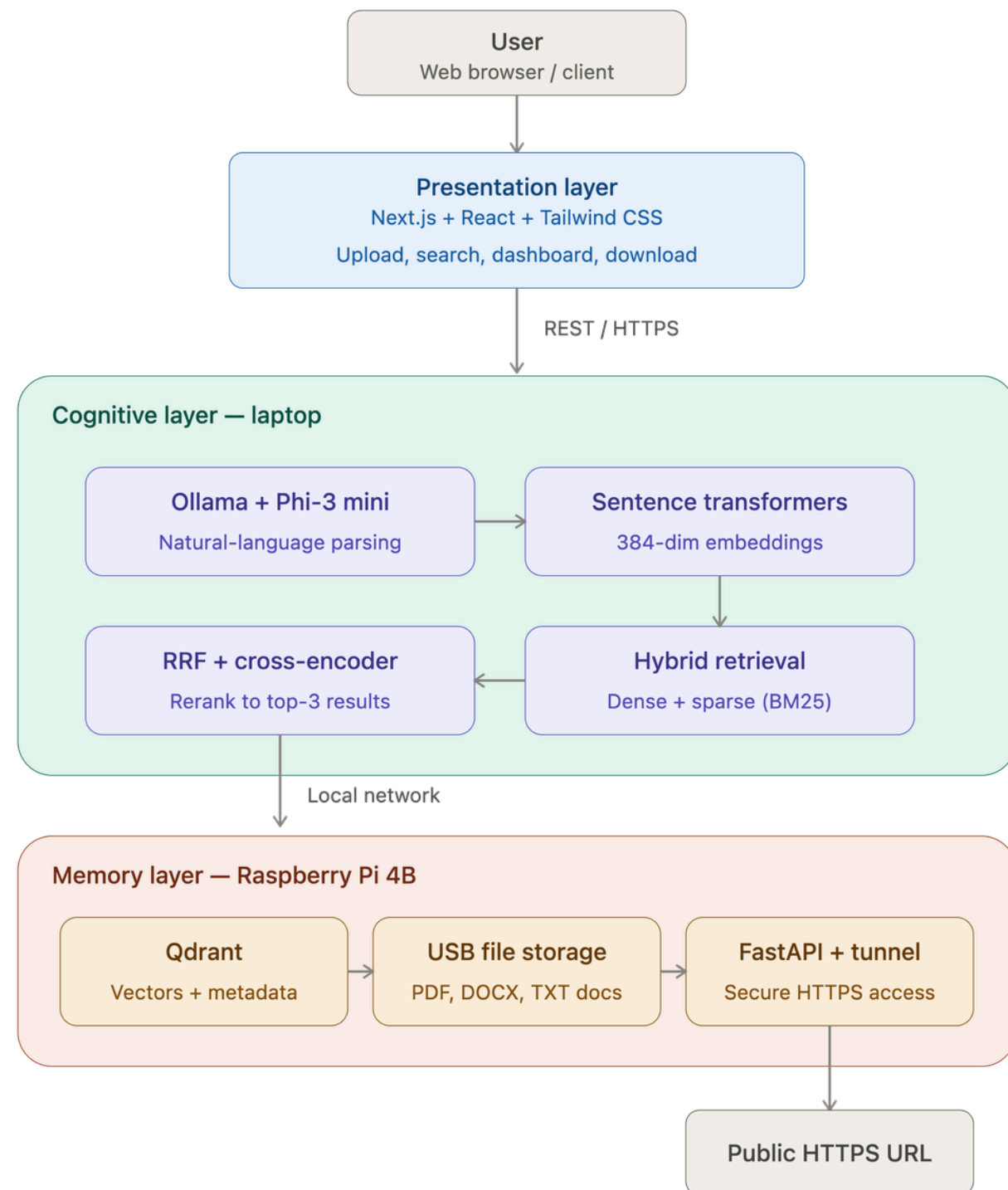


Table of Contents

- ❶ System Architecture
 - Modules & Components
 - Workflow / Process
 - Database / Index Design
 - Technology Stack
 - Hardware Requirements
 - Software Requirements
 - Algorithms, Tools & Frameworks
 - Implementation Plan
 - Timeline & Milestones
 - Task Allocation



System Architecture



Key Design Principle

Computation is performed on the laptop (Cognitive Layer), while persistent storage and always-on serving are handled by the Raspberry Pi (Memory Layer).

Architecture Characteristics

- Privacy-preserving: local intent parsing, embeddings and retrieval
- Hybrid retrieval: semantic + keyword search with RRF and reranking
- Self-hosted: no third-party LLM API calls
- Edge storage: Qdrant + USB persistent storage on Raspberry Pi
- Remote access: Cloudflare Tunnel provides secure HTTPS access



Detailed Description of Modules / Components

Software & Processing Modules

- Frontend - Home, Upload, Search and Dashboard for document interaction and system monitoring.
- Upload & Validation - Validates PDF/DOCX/TXT files, assigns a unique ID and initiates indexing.
- Text Extraction - Extracts document text using PyMuPDF/python-docx, with Tesseract OCR fallback for scanned files.
- Embedding & Indexing - Chunks text, generates 384-D MiniLM embeddings and stores vectors with metadata in Qdrant.
- Intent & Retrieval Engine - Phi-3 Mini interprets queries; dense + sparse retrieval, RRF and cross-encoder reranking produce the final Top-3.
- File & Metadata Manager - Maintains document metadata and connects search results to the original stored file.



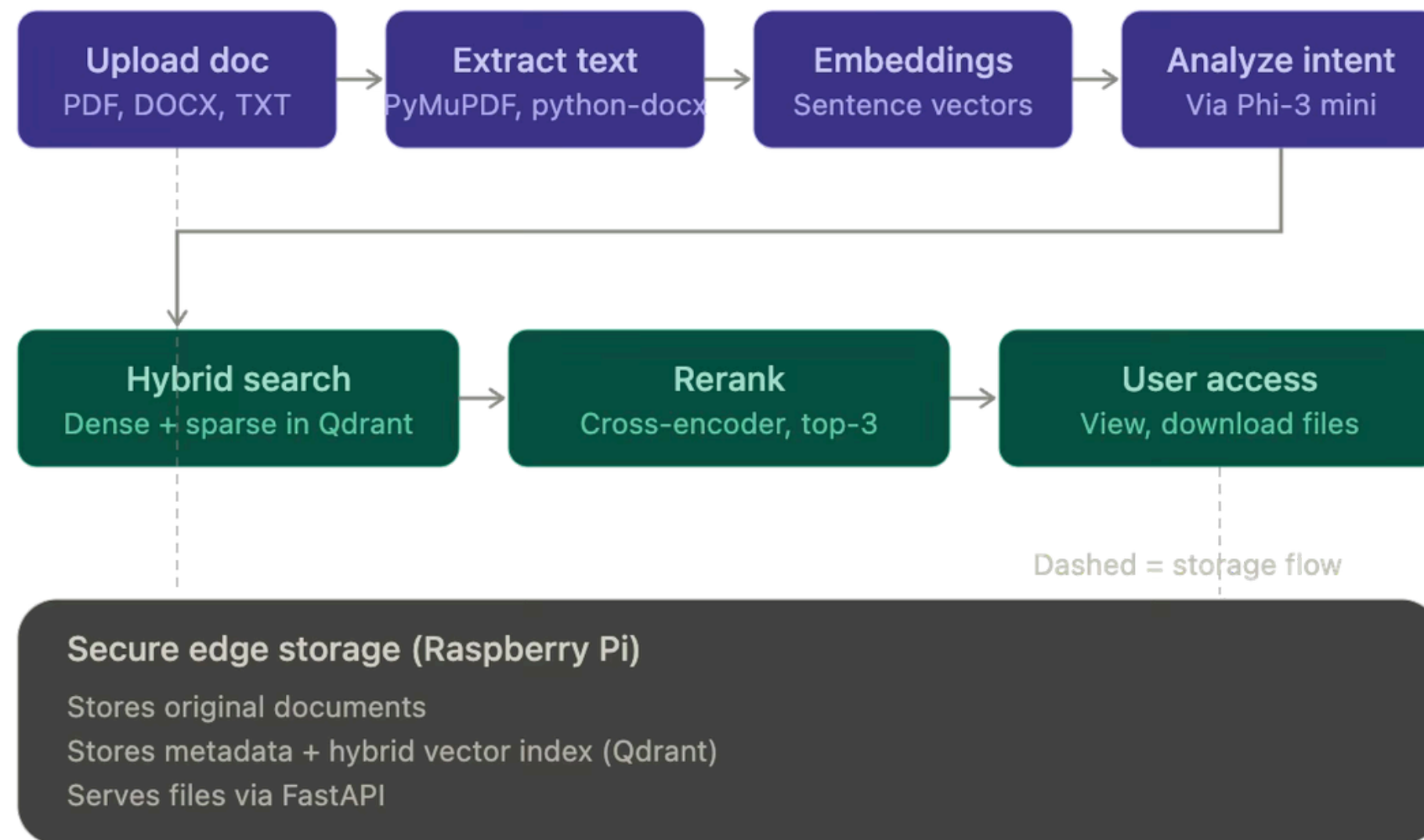
Detailed Description of Modules / Components

Edge/IoT Components

- Raspberry Pi 4 Model B (8 GB) - Memory-layer node hosting Qdrant, file storage and always-on serving.
- USB Storage (128 GB) - Persistent storage for original uploaded documents.
- MicroSD Card (64 GB) - Raspberry Pi OS boot and system storage.
- Cooling Case + Heatsinks - Provides thermal management during sustained operation.
- Cat6 Ethernet Cable - Provides a stable local connection between laptop and Raspberry Pi.
- ESP32 + 0.96" SSD1306 OLED - Monitoring interface used to visualize system status or processing flow
- Supporting Components: Breadboard and jumper wires for ESP32–OLED connections.



Workflow/Process Diagram



1. Document Ingestion

Uploaded PDF/DOCX/TXT files are validated, text is extracted, and document content is converted into semantic embeddings for indexing.

2. Query Processing & Retrieval

The user's natural-language query is interpreted by Phi-3 Mini. Intent-aware retrieval combines dense semantic search and sparse/BM25 search in Qdrant, followed by cross-encoder reranking to identify the most relevant Top-3 documents.

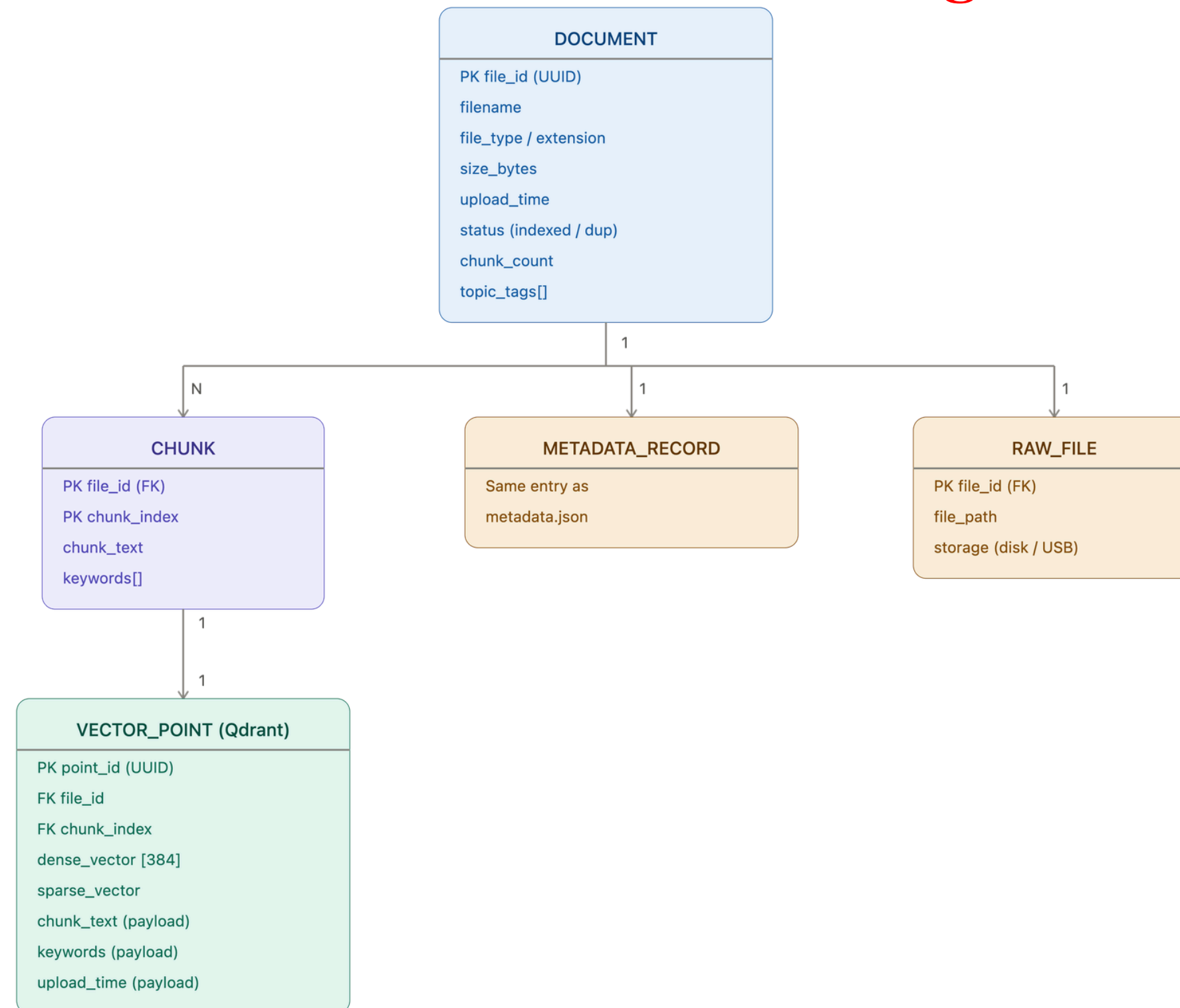
3. Secure Storage & Access

Original documents, metadata and the vector index are maintained on the Raspberry Pi edge node. The system returns the matched snippet and provides access to the original file through FastAPI

Document ingestion → Query processing → Retrieval → User access



Database Design & Data Management



Data Management & Design Controls

- Normalization: Document-level metadata stored once in metadata.json; original file stored once on disk/USB.
- Integrity: **'file_id'** is the common key across Document, Chunk, Qdrant Point and Raw File; duplicate similarity ≥ 0.95 is flagged.
- Security: Original data remains on local/Pi storage; remote access through Cloudflare HTTPS.
- Scalability: Chunk-based indexing and Qdrant support the target 150–200+ documents.



Dayananda Sagar University

School of Engineering

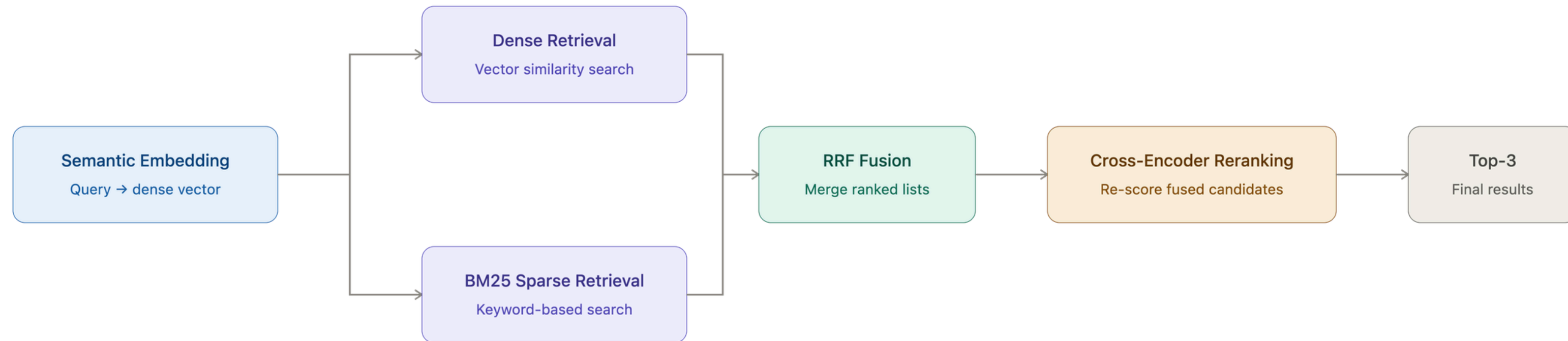
Technology Stack



Layer	Technology	Purpose
Frontend	Next.js + React + Tailwind CSS	Upload, search, dashboard and result UI
Backend	FastAPI + Uvicorn + Python 3.11	REST APIs and service orchestration
Intent Understanding	Ollama + Phi-3 Mini	Local natural-language intent parsing
Embeddings	Sentence-Transformers + MiniLM-L6-v2	384-D semantic representation
Vector Database	Qdrant	Persistent dense + sparse hybrid retrieval
Extraction / OCR	PyMuPDF + python-docx + Tesseract	PDF/DOCX/TXT and scanned-document processing
Reranking	Cross-Encoder (MS-MARCO MiniLM)	Reorders candidates for Top-3 relevance
Remote Access	Cloudflare Tunnel	Secure HTTPS without port forwarding
Edge Platform	Raspberry Pi 4B + USB	Local persistent memory and file serving



Core Algorithms



Why This Stack

- Performance: Lightweight local models + Qdrant hybrid search support consumer-grade hardware.
- Privacy: Intent parsing, embeddings and retrieval remain local with no third-party LLM API calls.
- Cost & Feasibility: Open-source components; no recurring AI API cost.
- Scalability: Chunk-based indexing and Qdrant support the target 150–200+ document corpus.
- Portability: Runtime supports Apple MPS / NVIDIA CUDA / CPU fallback.



Dayananda Sagar University

School of Engineering



Hardware & Software Requirements

Hardware Requirements	Software Requirements
Raspberry Pi 4 Model B - 8 GB Edge/memory layer for Qdrant, FastAPI and file serving	Operating System: Raspberry Pi OS 64-bit (Bookworm)
128 GB USB 3.0 Storage Persistent storage for original documents	Runtime: Python 3.11+ • Node.js 20 LTS
64 GB microSD (A2/U3) Boot and Pi OS storage	Backend: FastAPI • Uvicorn
USB-C 5.1 V / 3 A Power Supply Continuous Pi operation	AI Runtime: Ollama • Phi-3 Mini • PyTorch
Cooling Case + Heatsinks Thermal management for sustained operation	Embeddings: Sentence-Transformers • MiniLM-L6-v2
Cat6 Ethernet Cable Stable laptop ↔ Raspberry Pi connection	Vector Database: Qdrant (Embedded)
ESP32 + 0.96" SSD1306 OLED monitoring display	Document Processing: PyMuPDF • python-docx • Tesseract OCR
Breadboard + Jumper Wires ESP32–OLED prototyping connections	Remote Access: Cloudflare Tunnel (cloudflared)
	Development: Git / GitHub • macOS / Windows / Linux



Dayananda Sagar University

School of Engineering



Implementation Plan, Timeline & Task Allocation

Phase	Timeline	Deliverable
1. Data Ingestion & Preprocessing	Weeks 1–2	Upload, extraction & OCR fallback
2. Semantic Representation & Indexing	Week 3	Chunking, embeddings & Qdrant indexing
3. Intent-Aware Query Understanding	Week 4	Phi-3 Mini intent parsing + Review-1
4. Hybrid Retrieval & Reranking	Week 5	BM25, RRF & cross-encoder reranking
5. Secure Edge Deployment	Week 6	Raspberry Pi, USB storage & Cloudflare Tunnel
6. Evaluation & Validation	Weeks 7–9	Baselines, accuracy & latency evaluation



Dayananda Sagar University

School of Engineering



Implementation Plan, Timeline & Task Allocation

Development Approach

Laptop-first development → Incremental integration → Regression testing → Raspberry Pi migration → Quantitative evaluation

Key Milestones

- 29 Aug 2026 — Review-1: System Architecture & Design
- 12 Sep 2026 — Review-2: 25% Implementation Demo
- 22 Sep 2026 — Review-3: 50% Implementation, Results & Discussion

Task Allocation

Member	Assigned Responsibility
Sujith Putta	Backend development • System integration • Intent/search pipeline • Architecture & workflow design • Presentation
K Vikas Aneesh Reddy	Qdrant indexing • Duplicate detection • Retrieval testing • Reranking integration
Mokshith Karnati	Next.js frontend • Search interface • Dashboard UI



References

- [1][Krishnan, S., & Subramanian, N., "Semantic Search in a Document Management System Using Retrieval Augmented Generation," IEEE/ACIS SNPD, 2025.](#)
- [2][Jacob, T. P., Bizotto, B. L. S., & Sathiyarayanan, M., "Constructing the ChatGPT for PDF Files with LangChain – AI," IEEE International Conference on Inventive Computation Technologies \(ICICT\), 2024.](#)
- [3][Sawarkar, K., Mangal, A., & Solanki, S. R., "Blended RAG: Improving RAG Accuracy with Semantic Search and Hybrid Query-Based Retrievers," arXiv, 2024.](#)
- [4][Puspitasari, F. D., et al., "Efficient RAG with Intent-Aware Retrieval and Semantics-Preserving Chunking," arXiv, 2026.](#)
- [5][Nan, Y., et al., "Research on an Intelligent File Query System Model Integrating Natural Language Understanding and Information Retrieval," IEEE International Conference on Power, Electronics and Green Energy \(ICPEGE\), 2026.](#)



Thank You